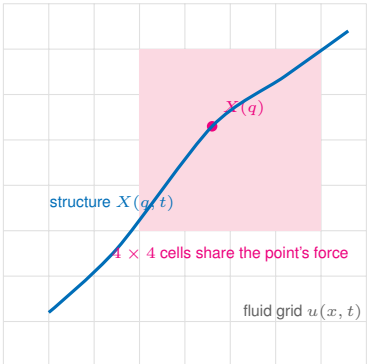
THE IMMERSED BOUNDARY METHOD AND NUMERICAL CONSIDERATIONS

What IBAMR solves, how it couples the fluid to the structure, and why runs blow up

Laura A. Miller

Department of Mathematics, University of Arizona

Using IB Software • Part III • Chapter 5



Claude (Anthropic) was used to organize, combine, and format more than twelve years (2013–2026) of the instructor's tutorial notes from courses and workshops at several universities.

LEARNING GOALS

By the end of this chapter you should be able to:

1. write down the equations that an IB simulation solves and say what each term does;
2. explain how the fluid grid and the structure talk to each other through regularized delta functions, and what that implies for point spacing and the thickness of a boundary;
3. use the words convergence, order and stability correctly, and show each with the two demo scripts;
4. say what limits the time step in an IB simulation and read the error IBAMR prints when the limit is crossed;
5. explain what adaptive mesh refinement does in IBAMR and which input keys control the time step, the grid and the coupling.

> This chapter is the practical overview. The derivations are in the companion series *Fluid Dynamics and IBM Theory*, modules 0-1 (Navier–Stokes), 0-2 (non-dimensionalization) and 1-1 to 1-4 (the IB method, its formulation, the integral transforms and the discretization).

FLUID–STRUCTURE INTERACTION IN BIOLOGY

Organisms move fluid and are moved by it, from the heart of a whale to a swimming bacterium. The same equations hold; what changes is the balance between inertia and viscosity, the Reynolds number.

Regime	Examples in the old lectures	Examples in this series
High Re: inertia matters	the whale heart (a chambered pump); the stingray (flapping locomotion)	jellyfish bells, insect wings, the flow tank (IBFE/10)
Low Re: viscosity dominates	cilia in the respiratory tract (micropumps); *E. coli* (flagellar locomotion)	bristled wings (IB/04), the glycocalyx (IB/08), heart tubes

> The immersed boundary method handles both regimes with one code: an elastic structure in a viscous incompressible fluid, on a fixed grid. That is why one tutorial covers whale hearts and bacteria.

The 2014–2025 lectures opened with four photographs of these organisms; they were third-party images and are replaced here by the table.

MODELING THE FLUID: THE NAVIER–STOKES EQUATIONS

For a viscous, incompressible fluid with velocity $\mathbf{u}(\mathbf{x}, t)$ and pressure $p(\mathbf{x}, t)$,

$$\rho\left(\frac{\partial \mathbf{u}}{\partial t} + \mathbf{u} \cdot \nabla \mathbf{u}\right) = -\nabla p + \mu \Delta \mathbf{u} + \mathbf{f},$$
$$\nabla \cdot \mathbf{u} = 0.$$

- The first line is Newton's second law, $F = ma$, for a parcel of fluid: mass per volume ρ times acceleration equals the forces per volume.
- $-\nabla p$: the pressure pushes from high to low; $\mu \Delta \mathbf{u}$: viscosity smooths velocity differences; $\mathbf{f}$: the force per volume that the immersed structure puts on the fluid.
- The second line says the fluid is incompressible: what flows into a region flows out.

> ρ is the density and μ the (dynamic) viscosity: in `input2d` they are `RHO` and `MU`. In the IB examples they are in SI units (1000 kg/m^3 and 100 N s/m^2 for the rubber band); in the IBFE examples they are dimensionless.

Derivation: *Theory*, module 0-1.

THE NON-DIMENSIONAL FORM

Choose a characteristic length L (the length of a wing) and velocity V (the speed of the wing tip), and write $\mathbf{x} = L\,\mathbf{x}'$, $\mathbf{u} = V\mathbf{u}'$, $t = (L/V)\,t'$, $p = \rho V^2 p'$. Then

$$\frac{\partial \mathbf{u}'}{\partial t'} + \mathbf{u}' \cdot \nabla' \mathbf{u}' = -\nabla' p' + \frac{1}{Re}\,\Delta' \mathbf{u}' + \mathbf{f}', \qquad Re = \frac{\rho V L}{\mu}.$$

- One number, the Reynolds number, is left: the ratio of inertial to viscous forces.
- Everything else (density, viscosity, size, speed) enters only through Re and through the dimensionless force $\mathbf{f}' = \mathbf{f}L/(\rho V^2)$.
- IBAMR does not care whether your input is in meters and seconds or dimensionless, as long as every number is in the same system. Chapter 8b shows how to convert a model.

> A choice to make early: the IB examples here keep SI units because the structure files were written that way; the IBFE examples are dimensionless because IBAMR's own examples are. Mixing them is the most common way to get a Reynolds number wrong by a factor of a thousand.

Derivation: *Theory*, module 0-2.

THE REYNOLDS NUMBER

- Flows with the same Re (and the same geometry and boundary conditions) are dynamically similar: the same dimensionless velocity field, the same force coefficients.
- That is how physical models work: a larger model of a small organism in a more viscous fluid (glycerin, oil), or a smaller, faster model of a large one.
- In a simulation the same freedom exists: a bristled wing a thousand times larger than the insect's, in a fluid a thousand times more viscous, gives the same flow. Use it to keep numbers near 1.

Example	Re and how it is built
IB/01 rubber band	very low ($\mu = 100$, $\rho = 1000$, $V \sim 0.1$, $L = 0.2$: $Re \approx 0.2$)
IB/03 tethered leaf	$\rho V \ell / \mu = 2.5$
IB/04 bristles	0.01 per bristle, 0.8 for the row
IB/08 glycocalyx (3D)	0.06
IBFE/10 flow tank (3D)	20 (speed 0.1, width 2, $\mu = 0.01$)

> Chapter 1b gave the comfortable ranges: IB2d up to about 10^3, IBAMR up to a few 10^3. Above that the boundary layers are thinner than the grid resolves, and the cost grows fast.

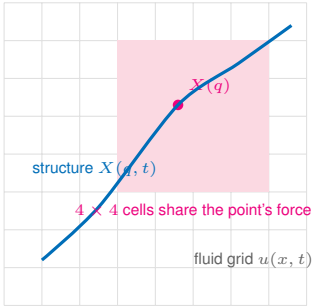
DESCRIBING THE STRUCTURE

In the immersed boundary method the structure is always elastic:

- it can be stiff enough to act like a rigid object, but it is still a collection of elastic elements;
- it can be moved in prescribed ways with target points, stiff springs between the structure and a path you choose (chapter 9a);
- it can resist stretching (springs), bending (beams), and twisting (rods), and it can carry mass (chapter 13);
- every one of these properties can change in time, by a formula or by another model (muscle activation, growth: chapters 9b and 20).

> In IBAMR the structure is a list of points in a `.vertex` file and lists of elements that join them: `.spring`, `.beam`, `.target`, `...` (chapter 6). In IBFE it is a finite element mesh with a material model (Part IV). Both produce the same thing the fluid needs: a force at every point.

THE IMMERSED BOUNDARY IDEA



Two descriptions, one problem:

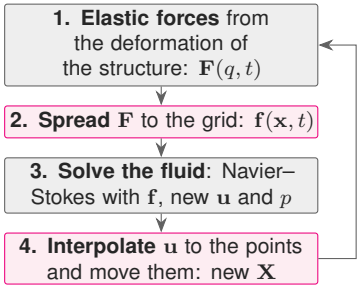
- Eulerian: the fluid lives on a fixed Cartesian grid. Unknowns at the grid points: velocity $\mathbf{u}(\mathbf{x},t)$, pressure $p(\mathbf{x},t)$, and the force density $\mathbf{f}(\mathbf{x},t)$ that the structure exerts.
- Lagrangian: the structure is a curvilinear mesh of material points labelled by q. Unknowns on the mesh: position $\mathbf{X}(q,t)$ and elastic force density $\mathbf{F}(q,t)$.

THE IMMERSED BOUNDARY IDEA: WHAT THE FIXED GRID BUYS

- The grid does *not* follow the structure, and the structure does not know about the grid. Each is discretized where it lives, and the two are coupled by integrals with delta functions.
- A body-fitted method would rebuild its mesh every time the structure moved; a jellyfish bell that folds, a heart valve that closes, or two wings that touch would each be a meshing problem of its own. Here the structure can do anything, including crossing itself, and the grid never changes.
- The structure can also be anything from a single point to a finite element solid; it only has to produce a force at each of its points.

> No body-fitted mesh, no remeshing when the structure moves: that is the whole point, and the price is a boundary a few cells thick (slide 16).

ONE TIME STEP



- Steps 1 and 4 happen on the structure, step 3 on the grid, and steps 2 and 4 are the coupling: the same regularized delta function spreads forces one way and interpolates velocities the other.
- Step 4 says that the structure moves with the fluid: the no-slip condition, enforced by construction.
- IBAMR's default scheme evaluates the forces and the velocity at the half step (the midpoint rule), which makes the coupling second-order accurate in time (slide 26).

The screen output of every example shows this loop: `At beginning of timestep`, the fluid solve's residuals, `integral{F}` (the sum of the spread forces), `At end of timestep`.

THE IB EQUATIONS

The Navier–Stokes equations with the structure's force, plus two coupling equations and an elasticity law:

$$\begin{aligned}
\rho\,(\mathbf{u}_t + \mathbf{u}\cdot\nabla\mathbf{u}) &= -\nabla p + \mu\Delta\mathbf{u} + \mathbf{f}, \qquad \nabla\cdot\mathbf{u} = 0,\\
\mathbf{f}(\mathbf{x},t) &= \int_\Omega \mathbf{F}(q,t)\,\delta\big(\mathbf{x}-\mathbf{X}(q,t)\big)\,dq,\\
\frac{\partial\mathbf{X}}{\partial t}(q,t) &= \mathbf{u}\big(\mathbf{X}(q,t),t\big) = \int \mathbf{u}(\mathbf{x},t)\,\delta\big(\mathbf{x}-\mathbf{X}(q,t)\big)\,d\mathbf{x},\\
\mathbf{F}(q,t) &= \mathcal{F}\big[\mathbf{X}(\cdot,t)\big](q).
\end{aligned}$$

Here $\mathbf{x}$ are the fixed physical coordinates, q the material coordinate of the structure (its arc length, say), Ω the set of material coordinates, and δ the Dirac delta function (in 2D, $\delta(x)\delta(y)$). *Theory*, modules 1-2 and 1-3, derive these equations and show that the transforms conserve total force and torque.

THE IB EQUATIONS, LINE BY LINE

- The first line is the Navier–Stokes system of slide 4 with one addition: the force density $\mathbf{f}$ that the structure exerts on the fluid. The fluid does not know what the structure is, only where it pushes.
- The second line converts the Lagrangian force density $\mathbf{F}$ (force per unit q, defined on the structure) into an Eulerian force density $\mathbf{f}$ (force per unit area or volume, defined on the grid): a point at $\mathbf{X}(q,t)$ puts its force at that position and nowhere else.
- The third line is the no-slip condition: each material point moves at the fluid velocity where it is. Written as an integral with the same δ, it is the transpose of the second line.
- The last line is the elasticity: $\mathbf{F}$ depends on the whole configuration $\mathbf{X}(\cdot,t)$, through springs, beams and target points (chapters 6, 9 and 10) or through a finite element stress (Part IV). It is the only line you write yourself.

WHAT THE DELTA FUNCTIONS MEAN

Integrate the force equation over any region R of the fluid:

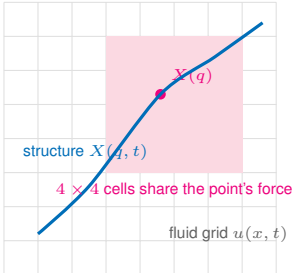
$$\int_R \mathbf{f}\, d\mathbf{x} = \int_\Omega \mathbf{F}(q,t) \underbrace{\int_R \delta\big(\mathbf{x} - \mathbf{X}(q,t)\big)\, d\mathbf{x}}_{1 \text{ if } \mathbf{X}(q,t)\in R,\ 0 \text{ otherwise}}\, dq,$$

so the force on the fluid in R is exactly the sum of the elastic forces of the material points that lie in R. Nothing is lost or created: the structure's force is simply handed to the fluid where the structure is.

The velocity equation reads the other way: the velocity of material point q is the fluid velocity at its position, $\mathbf{u}(\mathbf{X}(q,t),t)$. Writing it as an integral with the same δ is what lets the discrete version use the same kernel for both directions.

> Using the *same* regularized delta function for spreading and interpolation is not a convenience: it is what makes the discrete coupling conserve energy, so that the elastic energy stored in the structure equals the work done on the fluid. Peskin (2002) explains why.

SPREADING THE FORCE TO THE GRID



On the grid, with cell size h and points $\mathbf{X}_k$ carrying forces $\mathbf{F}_k$ (force per unit q, times the spacing Δq):

$$\mathbf{f}(\mathbf{x}_{ij}) = \sum_k \mathbf{F}_k \, \delta_h(\mathbf{x}_{ij} - \mathbf{X}_k) \, \Delta q,$$

with the regularized delta function $\delta_h(\mathbf{x}) = \frac{1}{h^2}\phi\left(\frac{x}{h}\right)\phi\left(\frac{y}{h}\right)$ (and a third factor in 3D).

- ϕ is zero beyond a few cells, so each point's force lands on a small block of cells around it: 4×4 for the 4-point kernel, with weights that add up to 1.
- A boundary that resists bending or stretching produces forces along it; the picture shows one point's share of the grid.

INTERPOLATING THE VELOCITY TO THE POINTS

The velocity of point k is the weighted sum of the velocities of the same block of cells:

$$\mathbf{U}_k = \sum_{ij} \mathbf{u}(\mathbf{x}_{ij})\, \delta_h(\mathbf{x}_{ij} - \mathbf{X}_k)\, h^2,$$

and the point moves with it:
$\mathbf{X}_k^{n+1} = \mathbf{X}_k^n + \Delta t\, \mathbf{U}_k$ (with the midpoint rule, the velocity at the half step).

- The weights are the kernel evaluated at the distance from the cell to the point, in cells; at a given step the velocity in every cell is known.
- The same ϕ, the same block of cells: the transpose of the spreading operation.

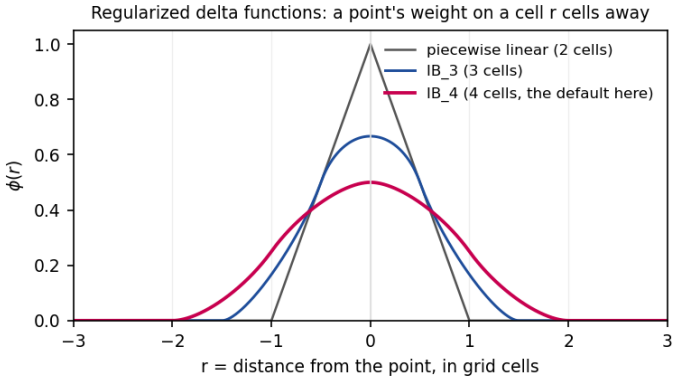
> Because the velocity is interpolated from a block of cells, a point never sees a velocity jump: the boundary is *smeared* over the kernel's width. Two surfaces closer than that width share the same fluid cells and cannot be told apart by the fluid (slide 16).

On an adaptive grid the points always live on the finest level, so h here is the finest cell size, $\Delta x = L/N_{\text{finest}}$ in the input files (slides 32–33).

REGULARIZED DELTA FUNCTIONS

Peskin's 4-point kernel, `IB_4`, is

$$\phi(r) = \begin{cases} \frac{1}{8}\left(3 - 2|r| + \sqrt{1 + 4|r| - 4r^2}\right), & |r| \le 1, \\ \frac{1}{8}\left(5 - 2|r| - \sqrt{-7 + 12|r| - 4r^2}\right), & 1 \le |r| \le 2, \\ 0, & |r| \ge 2. \end{cases}$$



WHAT A KERNEL MUST SATISFY

What a kernel ϕ should satisfy, and why:

- continuous, so that a point moving smoothly across the grid feels a smoothly changing velocity and exerts a smoothly changing force;
- bounded support (a few cells), so that each point touches only its neighborhood and the sums on slides 12 and 13 are cheap;
- the discrete moment conditions: $\sum_j \phi(r-j) = 1$ and $\sum_j (r-j)\,\phi(r-j) = 0$ for every r, so that total force and torque are conserved whatever the point's position relative to the grid;
- a constraint that makes the result nearly independent of where the point sits relative to the grid ($\sum_j \phi(r-j)^2$ constant), so that a structure does not feel the grid lines as it moves;
- for the 4-point kernel, the extra condition that the even and the odd grid points each receive half the weight, which lets it work with the staggered grid.

> Peskin (2002), section 6, derives `IB_4` from exactly these conditions; *Theory*, module 1-3, works through it. The figure on the previous slide is plotted from the formulas, with IBAMR's names.

THE KERNELS IBAMR OFFERS

delta_fcn	cells	notes
IB_3	3	Roma, Peskin and Berger's 3-point kernel
IB_4	4	Peskin's classic; used by every example here
IB_5, IB_6	5, 6	smoother, more nearly grid-independent (Bao, Kaye and Peskin 2016); each point touches more cells
IB_4_W8	8	a wider 4-point-type kernel
PIECEWISE_LINEAR, PIECEWISE_CUBIC	2, 4	interpolation-type kernels
BSPLINE_3 ... BSPLINE_6	3–6	B-splines
COMPOSITE_BSPLINE_32, ...		B-splines with different widths for spreading and interpolation
DISCONTINUOUS_LINEAR, USER_DEFINED		special cases

The names are the strings that IBAMR 0.19.0 accepts, from

`ibtk/src/lagrangian/LEInteractor.cpp`. The "cells" column is the width of the stencil in each direction: a point touches that many cells across, squared in 2D and cubed in 3D.

CHOOSING THE KERNEL

```
IBMethod {
   delta_fcn = DELTA_FUNCTION  // "IB_4"
   enable_logging = ENABLE_LOGGING
}
```

The key is `delta_fcn` in the `IBMethod` block; our inputs set it from the variable `DELTA_FUNCTION` at the top of the file, next to the physical parameters.

- A wider kernel smooths more (a thicker boundary, fewer grid-position artifacts) and costs more; a narrower one is sharper but leaks and jitters more.
- Keep `IB_4` unless you have a reason; change it only together with the point spacing (slide 17), and rerun a convergence check afterwards, because the effective thickness of every structure changes with it.
- The 6-point kernel is the one to try when a structure at rest jitters or drifts with the grid; the 3-point kernel when a thin gap must stay open and refining is too expensive.

THE BOUNDARY IS A FEW CELLS THICK

- A point's force is spread over the kernel's width, and its velocity is averaged over the same width: to the fluid, an infinitely thin membrane looks like a layer about 4 cells thick with `IB_4`.
- The layer is not a flaw to be removed but the mechanism itself: the force has to reach the grid somewhere, and the kernel decides where. A sharper kernel gives a thinner layer and more grid artifacts; a wider one the reverse (slide 15).

> Rule: nothing in an IB model is thinner than about 4 cells, and two surfaces closer than that are one surface to the fluid. If a gap matters (a valve, a channel), resolve it with at least 4 to 8 cells, which may mean another refinement level (slide 32).

THE THICK BOUNDARY: THREE EXAMPLES FROM THIS SERIES

Consequences you will meet in the examples:

- **IB/07** moving plates: the plates start 5 mm apart, 2.6 cells, so at first they push and pull the same cells; the fluid between them is trapped and the force to pull them apart is enormous (63,000 N) until the gap is a few cells wide.
- **IB/04** bristles: each bristle is a ring of 33 points 5 cells across; with a kernel 4 cells wide the ring is nearly a solid disk, which is what we want.
- **IB/08** glycocalyx: the fiber's radius was reduced by $1.25\,\Delta x$ in the generator "to allow for the width of the delta function", so that the fiber the fluid sees has the intended radius.

> Each README gives the numbers behind these statements and the input settings to reproduce them.

THE THICK BOUNDARY IN IBFE, AND WHAT TO DO ABOUT GAPS

- The IBFE method spreads forces from quadrature points inside the elements with the same kernel, so the same rule holds for thin finite element plates: IBFE/01's plate is 0.031 thick on a 0.016 grid, 2 cells, and the fluid sees it as a layer about 4 cells thick.
- When a gap must be open (a valve that leaks, a channel that must carry flow), count its width in cells on the finest level. Fewer than about 4 and it is closed; 4 to 8 and it carries a reduced flow; more and it behaves like a gap.
- The remedies are a finer finest level (another level in `MAX_LEVELS`, which costs time only near the structure), a narrower kernel (`IB_3`, with its own grid artifacts), or a larger model (the Reynolds number does not change if you scale length and viscosity together, slide 6).
- The reverse also matters: two structures meant to touch (the plates of IB/07, the two corals of chapter 21) never do; a thin film of fluid stays between them, and the force needed to squeeze it out is large.

POINT SPACING

- The classic rule for the IB method: about two points per grid cell along the boundary, $\Delta s \approx \Delta x/2$ on the finest level. Closer spacing changes nothing (the kernels overlap anyway); wider spacing lets fluid leak between points.
- Every IB example in this series follows it: the rubber band's 644 points are $0.55\,\Delta x$ apart, the leaf's 103 points $0.5\,\Delta x$, the bristles' rings $0.5\,\Delta x$, the glycocalyx rings about $\Delta x/2$ in both directions.

When you refine the grid, refine the structure with it: the generators take N (the finest grid count) and compute Δs from it, so one change to N keeps $\Delta s \approx \Delta x/2$. The stiffness constants in the files are scaled by Δs too (chapter 6), which is why the generators, not the files, are the thing to edit.

POINT SPACING FOR IBFE, AND CHECKING IT

- The 2020 lecture noted that “this may not be the best choice now; IBFE uses 1:1”. For IBFE the rule is stated in element sizes: elements about one cell across in our examples (IBFE/05: $lc = \Delta x$, `IB_POINT_DENSITY = 1.0`); IBAMR’s own `IBFE/explicit/ex4` uses elements two cells across with `IB_POINT_DENSITY = 3`, which puts about the same number of quadrature points per cell.
- `IB_POINT_DENSITY` sets how many quadrature points per cell the elements contribute; it is the IBFE equivalent of Δs, and chapter 17 shows what changing it does.
- `test_IB2d_IBAMR_Geometry.m` (Appendix A, with its Python version) plots any structure file with its point spacing, so you can check an IB structure before running; for a finite element mesh, chapter 28 shows how to look at it in VisIt with the grid.

> Two points per cell for IB, one element per cell for IBFE: both put the force on the grid at about the grid’s own resolution, which is all the kernel can use.

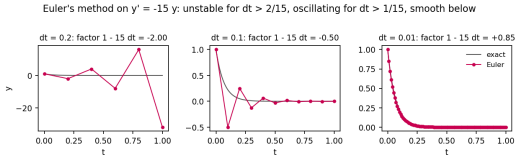
NUMERICAL ANALYSIS: THREE WORDS

Numerical analysis designs methods and also analyzes them. Three ideas carry most of the weight:

- Convergence: as the step sizes (Δx, Δt) go to zero, the numerical solution approaches the exact one.
- Order: how fast. An error proportional to h^2 is second order: halving h divides the error by 4.
- Stability: whether errors (rounding, truncation) are damped or amplified. An unstable method can be "convergent in principle" and still give garbage, because the errors grow exponentially before h is small enough.

> Everything that goes wrong in an IB run is one of these three: an *unstable* time step (the run blows up), a grid that has *not converged* (the answer changes when you refine), or an *order* so low that refining is hopeless. The next slides show each on a problem small enough to see.

INSTABILITY: A STIFF EQUATION AND EULER'S METHOD



Solve $y' = -15\,y$, $y(0) = 1$, whose exact solution e^{-15t} decays smoothly, with Euler's method: $y_{n+1} = y_n + \Delta t\,(-15\,y_n) = (1 - 15\Delta t)\,y_n$.

- Each step multiplies y by $1 - 15\Delta t$. For $\Delta t = 0.2$ that factor is -2: the numerical solution grows and changes sign every step. Unstable.
- For $\Delta t = 0.1$ the factor is -0.5: it decays, but oscillates in sign. Stable, wrong in shape.
- For $\Delta t = 0.01$ the factor is 0.85: smooth decay, close to the exact curve.

> Stable for $\Delta t \le 2/15$, well-behaved for $\Delta t < 1/15$. The stiffer the equation (the larger the 15), the smaller the step. Springs in an IB model are exactly this kind of term.

TRY IT: THE STIFF EQUATION

In MATLAB or Octave, from `examples/support/matlab-exercises`:

```
stiff_ode_example(0.2, 1)    % unstable
stiff_ode_example(0.1, 1)    % stable, oscillates
stiff_ode_example(0.01, 1)   % smooth
```

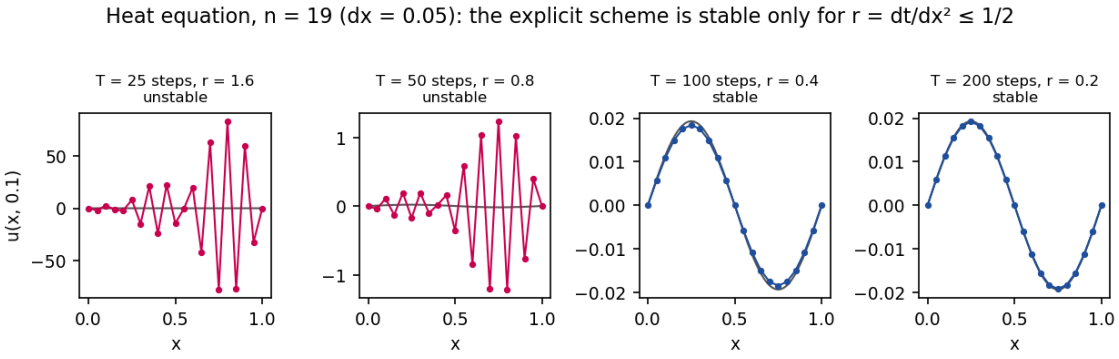
In Python (the same plots, saved as `stiff_ode_example.png`; add `--show` for a window):

```
python3 stiff_ode_example.py 0.2 1
python3 stiff_ode_example.py 0.1 1
python3 stiff_ode_example.py 0.01 1
```

- The arguments are the time step and the final time.
- Find the largest stable step by halving and doubling; then check the claim $\Delta t \leq 2/15$ with $\Delta t = 0.13$ and 0.14.
- Change -15 to -150 in the script. What happens to the stable step?

The scripts are the course's, with two bugs fixed: the MATLAB version allocated `zeros(N)` (an $N \times N$ matrix) instead of a vector and stopped one step before `tfinal`. The Python version writes the same numbers (Appendix A).

STABILITY IN A PDE: THE HEAT EQUATION



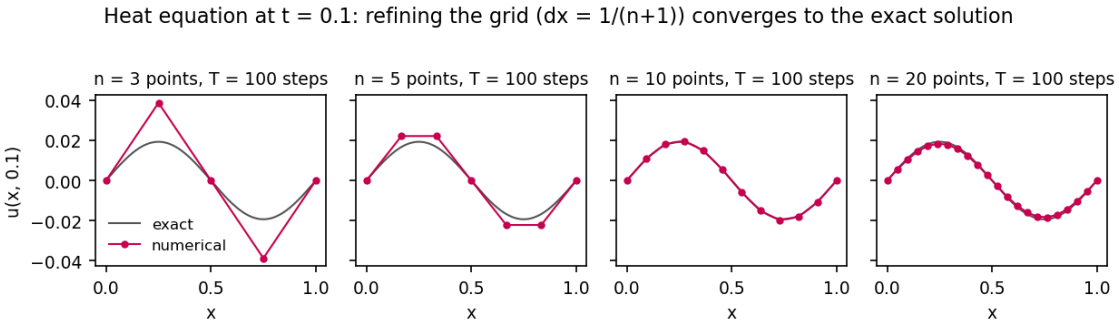
$u_t = u_{xx}$ on $0 < x < 1$ with $u = 0$ at the ends and $u(x, 0) = \sin 2\pi x$: the temperature of a bar with a hot and a cold spot, which diffuse away. Exact solution $e^{-4\pi^2 t} \sin 2\pi x$. The explicit scheme $u_j^{n+1} = u_j^n + r\,(u_{j+1}^n - 2u_j^n + u_{j-1}^n)$ with $r = \Delta t/\Delta x^2$ is stable only for $r \leq 1/2$.

THE HEAT EQUATION: WHAT THE FOUR PANELS SHOW

- With $n = 19$ interior points ($\Delta x = 0.05$) and T steps to $t = 0.1$: $T = 25$ and 50 ($r = 1.6$ and 0.8) blow up (to about 10^2 and 10^0 by $t = 0.1$, and without bound after that), while $T = 100$ and 200 ($r = 0.4$ and 0.2) follow the exact curve.
- The growing mode is the most oscillatory one the grid can hold, $(-1)^j$, which is amplified by $|1 - 4r|$ per step: 5.4 at $r = 1.6$, 2.2 at $r = 0.8$. That is the zigzag in the unstable panels.
- Just above $r = 1/2$ the growth starts from rounding error and takes many steps to show: with $T = 70$ ($r = 0.57$) nothing is visible yet at $t = 0.1$. Instability is exponential, but it needs time.
- Halving Δx forces Δt down by four: the price of an explicit diffusion step, and the reason IBAMR treats viscosity implicitly.

> An IB run that blows up does the same thing on its own grid: a short-wavelength mode in the structure or the velocity grows by a fixed factor per step until the numbers overflow, and a smaller step is the only cure.

CONVERGENCE IN SPACE



With $T = 100$ steps (always stable here) and $n = 3, 5, 10, 20$ points, the numerical solution approaches the exact curve: convergence in space. The scheme is second order in Δx: doubling n divides the error by about 4. Measure it yourself: the error at $x = 1/4$ for $n = 5, 10, 20$.

CONVERGENCE IN SPACE: WHAT IT MEANS FOR AN IB RUN

- For an IB simulation the question is the same: is the flow around the boundary resolved? Are the shed vortices resolved? Is the gap between two structures resolved?
- Chapter 16 makes this a procedure: run on two or three grids (N, $2N$, $4N$ on the coarsest level, which refines every level), compare a number you care about (a drag force, a swimming speed, a tip position), and stop refining when it changes by less than you can accept.
- Convergence in time is checked the same way with Δt; in IB runs it is usually the space resolution that limits accuracy, because the time step is already small for stability (slide 27).
- Chapter 4d does the whole exercise in IB2d on a laptop, where a grid study takes minutes; the same study in IBAMR is the first thing to do before a long 3D run.

> A converged answer on a coarse grid is worth more than an unconverged one on a fine grid, because you know what it means.

TRY IT: THE HEAT EQUATION

```
heat_equation(n, T)   % n interior points, T steps to t = 0.1
heat_equation(3, 100), heat_equation(5, 100)
heat_equation(10, 100), heat_equation(20, 100)  % space
heat_equation(19, 25), heat_equation(19, 50)    % unstable
heat_equation(19, 100), heat_equation(19, 200)  % stable
```

```
python3 heat_equation.py 19 100    # dx = 0.05, r = 0.4
python3 heat_equation.py 19 50     # r = 0.8: blows up
python3 heat_equation.py 19 70     # r = 0.57: not yet visible
```

- The plot shows the numerical and the exact solution at $t = 0.1$; the Python version prints Δx, Δt and r and saves `heat_equation.png`.
- The 2018 lecture used $n = 10$ with $T = 2, 4, 8, 16, 32, 64$. With so few points the unstable runs (any $T < 25$) have not yet grown visibly by $t = 0.1$, and the solutions look merely wrong; $n = 19$ with $T = 25$ and 50 shows the real blow-up.
- The course's `heat_equation.m` had a header that said $\sin 3\pi x$ while the code used $\sin 2\pi x$, and wrote an EPS file on every call; both are fixed, and the Python version gives identical numbers (checked with Octave).

RATE AND ORDER OF CONVERGENCE

For a sequence $x_k \to L$:

- it converges linearly if $|x_{k+1} - L| \le \mu\,|x_k - L|$ for some $0 < \mu < 1$; μ is the rate;
- it converges with order $q > 1$ if $|x_{k+1} - L| \le C\,|x_k - L|^q$: quadratic for $q = 2$, cubic for $q = 3$ (Newton's method is quadratic).

For a discretization with step h the error behaves like $E(h) \approx Ch^p$, and p is the order of accuracy. From two runs,

$$p \approx \frac{\log\big(E(h_1)/E(h_2)\big)}{\log(h_1/h_2)},$$

and when the exact solution is unknown, three runs with h, $h/2$, $h/4$ give p from the differences between successive solutions.

Chapter 16 does this with IBAMR: the same example on grids with N, $2N$ and $4N$ cells, a drag force or a tip position measured in each, and p from the formula. Expect p between 1 and 2 for an immersed boundary (next slide).

HOW ACCURATE IS THE IB METHOD?

- The older lectures said “second order in space if the boundary is smooth, first order if it has edges”. That is too generous.
- The fluid solver and the time stepping are second order, and the method is formally second order when everything is smooth, for example a smooth body force with no interface.
- But a thin elastic boundary makes the pressure jump and the velocity gradient jump across it, and the regularized delta function smears those jumps over a few cells. Near the boundary the error is then first order in Δx (Griffith and Peskin 2005); away from it, second order.

> So: refine until the quantity you care about stops changing, and say on what grid it was computed. Do not expect halving Δx to divide the error by four near an immersed boundary.

ACCURACY IN PRACTICE

- In practice the numbers you report (forces, speeds, positions) converge at a rate between 1 and 2, and the resolution of boundary layers and shed vortices matters more than the formal order.
- Quantities that live on the structure (a tip position, a swimming speed) converge faster than quantities that need the jump across it (the pressure difference across a membrane, the local shear stress), because the smearing of the kernel averages the latter over a few cells.
- The same holds for IBFE: the solid's interior is second order, but the fluid–solid interface is smeared in the same way (Griffith and Luo 2017 discuss the trade-offs).
- When you compare with an experiment or a body-fitted code, compare integrated quantities first (drag, lift, flow rate), then fields away from the boundary, and only last the fields at the boundary.

B. E. Griffith and C. S. Peskin, *On the order of accuracy of the immersed boundary method: higher order convergence rates for sufficiently smooth problems*, J. Comput. Phys. 208 (2005) 75–105. B. E. Griffith and X. Luo, *Hybrid finite difference/finite element immersed boundary method*, Int. J. Numer. Meth. Biomed. Engng 33 (2017) e2888.

TIME STEPPING IN IBAMR

- The IB coupling (`IBExplicitHierarchyIntegrator`) uses the midpoint rule by default: forces and velocities are evaluated at $t + \Delta t/2$, which is second order in time. The trapezoidal rule is the other option.
- The fluid (`INSStaggeredHierarchyIntegrator`) uses a staggered-grid projection method: viscosity implicit, the convective term explicit with Adams–Bashforth (second order) and the PPM limiter, as in our inputs: `CONVECTIVE_TS_TYPE = "ADAMS_BASHFORTH"`, `CONVECTIVE_OP_TYPE = "PPM"` (alternatives: `CUI`, `CENTERED`, `UPWIND`, `STABILIZED_PPM`).
- The pressure is found by a projection: the velocity is advanced without the pressure gradient and then corrected so that $\nabla \cdot \mathbf{u} = 0$; the residuals of that solve are the `stokes solve` lines in the log (chapter 3).

> The older notes said “first or second order in time, depending on the version”. With IBAMR 0.16 and 0.19 and these inputs it is second order. The explicit coupling is also why the time step is limited by the stiffest force in the model (next slides).

TIME STEPPING IN IBAMR: THE FIRST STEP, AND THE ALTERNATIVES

- The first step has no history for Adams–Bashforth, so the integrator takes two cycles of fixed-point iteration; that is the message `attempting to perform 2 cycles of fixed point iteration` at the start of every run, and the reason a predictor–corrector advection–diffusion solver needs `init_convective_time_stepping_type = "FORWARD_EULER"` (IBFE/12's README).
- A restarted run also starts with two cycles; the skip of the time-step-change check on the first step and after a restart is in the same place in the source (`IBHierarchyIntegrator.cpp`).
- `num_cycles` in the integrator block asks for more cycles per step at every step; the course inputs use 1, and the examples here keep it.
- IBAMR also has implicit and semi-implicit IB integrators for very stiff structures, and a collocated-grid fluid solver (`SOLVER_TYPE = "COLLOCATED"` in our inputs switches to it); chapter 15 looks at the examples that use them, and chapter 23 walks through the integrator calls in a main file.

WHAT LIMITS THE TIME STEP: THE MEASURED FACTS

Example	Stiffest element	Δx (m)	Largest DT that runs (s)	What the next larger step does
IB/01 rubber band	springs $K/\Delta s = 2.0 \times 10^6$	1/512	5×10^{-4}	10^{-3} aborts within a few steps
IB/03 tethered leaf	beams 5.3×10^6, targets 3.1×10^4	1/512	2×10^{-4}	5×10^{-4} aborts after 7 steps
IB/07 moving plates	targets 2.0×10^6 on every point	1/512	10^{-4}	2×10^{-4} aborts after 7 steps
IB/04 bristles	targets 4.1×10^6 on every point	1/1024	2×10^{-5}	5×10^{-5} aborts after 22 steps
IB/08 glycocalyx (3D)	targets 3.2×10^4 on 13,600 points	1/512	2×10^{-5}	5×10^{-5}: velocity grows tenfold per step, aborts after 4
IB/06 growing springs	springs 2.0×10^6	1/512	2×10^{-4}	5×10^{-4} aborts after 136 steps

- The older lecture said “the time step is inversely proportional to the force”. The measurements say it more usefully: the stiffest element sets the step, and a finer grid needs a smaller step too (the bristles, with stiffness scaled by Δs, need a step five times smaller on a grid twice as fine).
- For a single explicit spring of stiffness κ on a mass m the stable step is $\Delta t < 2\sqrt{m/\kappa}$; in a viscous fluid the drag changes the constant but not the trend: stiffer, smaller.
- None of these runs is anywhere near the CFL limit (next slide): the instability comes from the forcing, as the older lecture said.

WHAT THE BLOW-UP LOOKS LIKE, AND THE KEYS BEHIND IT

```
IBHierarchyIntegrator::preprocessIntegrateHierarchy():
  Time step size change encountered.
Aborting.
```

- Each step IBAMR computes the largest stable step from the velocities, $\Delta t_{\text{CFL}} = \texttt{cfl} \times \min(\Delta x / |u|)$ over all levels, and uses $\min(\texttt{dt_max}, \Delta t_{\text{CFL}})$.
- While the run is healthy, Δt_{CFL} is far larger than `dt_max` and every step is `DT`. When the forcing instability makes the velocity grow, Δt_{CFL} drops below `DT`, the step *would* change, and with `error_on_dt_change = TRUE` the IB integrator stops instead.

Chapter 3 shows the same message in its table of first errors, with the rubber band at `DT = 1.0e-3`.

THE SYMPTOM AND THE CAUSE

- So the message about the time step changing is the symptom; the cause is the stiff forcing. Lowering `cfl` or raising it changes nothing, because the CFL number was not the problem; what matters is the growth of the velocity near the structure.
- In 3D the same growth can trigger a regrid with absurd velocities first, and the run dies with `Internal RefineSchedule<DIM> error` instead (IB/08's README, at `DT = 5.0e-5`): the regrid asks for refinement where the exploding velocity is, in a place no coarser level covers.
- Both messages appear within a few steps of the start, or within a few steps of a change in the model (a stiffer spring, a faster target). A run that has gone on for thousands of steps and then stops with them has usually hit a new event: two structures meeting, a fold, a wall.

> The fix is never to allow the change: it is a smaller `DT`, softer elements, or a slower motion. Chapter 16 gives the order in which to try them.

THE KEYS BEHIND THE TIME STEP

Key (our variable)	Meaning
`dt_max` (`DT`)	the step you ask for
`cfl` (`CFL_MAX`, 0.3)	the CFL number allowed
`error_on_dt_change`	abort when the step would change (TRUE here)
`warn_on_dt_change`	warn instead of aborting
`grow_dt` (`GROW_DT`, 2)	how fast the step may grow after a reduction
`dt_init`, `dt_min`	the first step and the smallest allowed
`num_cycles`	fixed-point cycles per step (1)
`regrid_cfl_interval`	regrid after this many cells of motion (0.5)

- `dt_max` is the step you want; `DT` in our inputs is just the variable that feeds it (and `dt_max` of the fluid integrator too).
- `cfl` is the CFL number the fluid integrator allows; 0.3 is the value in every example here.
- `grow_dt` matters only when the step was reduced: it limits how fast the step may grow back. With `error_on_dt_change = TRUE` a reduction never happens, so it is never used.
- `warn_on_dt_change` instead of `error_on_dt_change` lets the run continue with a smaller step; IBAMR's own examples do this, but a run that needs it is a run that is blowing up.

THE CFL CONDITION

For an explicit scheme that transports something at speed $|u|$ on a grid with spacing Δx, information must not travel more than $C_{\max}$ cells per step:

$$C = \frac{|u|\,\Delta t}{\Delta x} \le C_{\max}, \qquad \text{typically } C_{\max} \le 1.$$

- Courant, Friedrichs and Lewy (1928): a necessary condition for the stability of explicit finite-difference schemes for hyperbolic equations; IBAMR's convective term is one.
- In IBAMR the check runs on every level, and the finest level is the binding one, since Δx is smallest there.

> The condition says nothing about accuracy: a step that satisfies it can still be far too large for the stiff forces (slide 27). It is a floor under the step, not a recommendation.

THE CFL CONDITION IN OUR RUNS

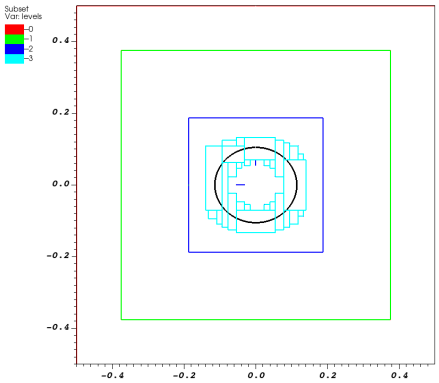
- In our runs C is tiny: the rubber band at 0.1 m/s with $\Delta t = 10^{-4}$ s and $\Delta x = 2$ mm gives $C = 0.005$. “In IBAMR we rarely have to worry about the CFL condition”, as the older lecture said, is true for the runs in this series, *until* something blows up (slide 28).
- When would it bind for real? Fast flows on fine grids: a prescribed motion at 1 m/s on a $1/1024$ m grid allows at most $\Delta t \approx 3 \times 10^{-4}$ s at `cfl = 0.3`, about the step the forcing allows anyway. Check it when you speed up a motion or refine the grid.
- The CFL number C is also what `regrid_cfl_interval` counts (slide 32): the number of cells a point may have moved since the last regrid. With 0.5 and $C = 0.005$ per step, the rubber band regrids about every 100 steps, which matches the regrid lines in its log.
- The log prints the CFL number when logging is on; `grep -i cfl IB2d.log` shows it over the run, and a value that climbs toward 0.3 is the early warning of a blow-up.

RESOLUTION: WHAT HAS TO BE RESOLVED

- The boundary layer on the structure: its thickness scales like $L/\sqrt{Re}$; at $Re = 2000$ and $L = 1$ that is about 0.02, so a grid of 512 cells across a box of width 2 puts about 5 cells in it. At $Re = 10^4$ it would be 2 cells.
- The shed vortices: a vortex needs several cells across its core, and it is advected for many steps, so a coarse grid both smears and misplaces it (chapter 1b's snake wake).
- The gaps between structures and the thinnest features of the structure itself: at least 4 to 8 cells (slide 16).
- The older notes said "typically 512 $\times$ 512". With adaptive refinement the finest level of our examples is 512 or 1024 cells across the box in 2D, and $64 \times 64 \times 512$ in 3D (IB/08), but only near the structure.

> The test is always the same: run on two grids and compare. If the number you care about changes by more than you can accept, refine again. Chapter 16 gives the procedure and the cost estimate; chapter 4d does it in IB2d, where it is cheap.

ADAPTIVE MESH REFINEMENT IN IBAMR



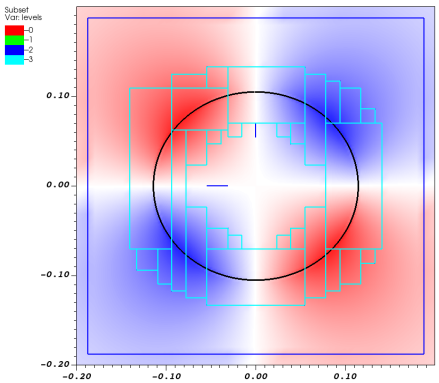
The rubber band's grid at $t = 0.12$ s: patch outlines of the four levels (example IB/01, IBAMR 0.19.0, rendered with VisIt).

- The domain is covered by a hierarchy of levels: level 0 is the coarsest grid (N cells across); each finer level has cells `REF_RATIO` times smaller (4 here) and covers only part of the domain, as a set of rectangular patches.
- Cells are tagged for refinement where the structure's points are, with `TAG_BUFFER` extra cells around them, and optionally where the vorticity is large (`VORTICITY_TAGGING`, `vorticity_rel_thresh`).

ADAPTIVE MESH REFINEMENT: WHERE THE STRUCTURE LIVES AND WHEN THE GRID CHANGES

- The structure always lives on the finest level (`level_number = MAX_LEVELS - 1` in the initializer block), so Δx at the structure is $L/(N \cdot \texttt{REF_RATIO}^{\texttt{MAX_LEVELS}-1})$: $1/512$ m for the rubber band (8×4^3) and for the leaf (32×4^2), $1/1024$ m for the bristles (16×4^3).
- The hierarchy is regridded when the points may have moved `regrid_cfl_interval` cells (0.5 here) or every `regrid_interval` steps, so the fine patches follow the structure. The rubber band regridded 163 times in its 20,000 steps; the moving plates, which travel fastest, 413 times in 10,000.
- At a regrid the patches are rebuilt, the data copied, and the pressure re-projected (the `regridProjection` lines in the log); each regrid costs about as much as a few time steps.
- A structure given a `level_number` that is not the finest, or that no longer exists after `MAX_LEVELS` is changed, is the most common way to get a run that is silently wrong (slide 37).

THE BASIC ADAPTIVE IB APPROACH, STEP BY STEP



Vorticity of the relaxing rubber band with the patch outlines of the finest levels (IB/01, $t = 0.12$ s).

The five points of the 2014 lecture, which later versions dropped:

1. Discretize the Eulerian equations on a Cartesian grid.
2. Refine the Cartesian grid near the boundary (and where the flow needs it).
3. Discretize the Lagrangian equations on a moving curvilinear mesh.
4. The curvilinear mesh is always at the same, finest, refinement level: every point sees the same Δx, and the kernel never straddles a level boundary.
5. Couple the two through regularized delta functions, as on a uniform grid.

WHAT IBAMR ADDS TO THE METHOD, AND WHERE THESE FIGURES COME FROM

- IBAMR is a parallel, adaptive implementation of exactly the method of slides 8–13; the adaptivity changes where the cells are, not what is done in them. The coupling, the kernel and the point spacing rules are those of the uniform-grid method.
- Because the structure is confined to the finest level, the delta function never has to be defined across a change of cell size; the coarser levels only carry the far field, and the finest level's cells are the h of slides 12 and 13.
- Griffith, Hornung, McQueen and Peskin (2007) describe the adaptive method and show that it keeps the formal second-order accuracy of the uniform-grid scheme for smooth problems.
- The 2014–2025 lectures illustrated this with three figures from B. Griffith's 2009 MBI workshop slides (the link, `mbi.osu.edu/2009/ctwmaterials/griffith.pdf`, is dead). They are replaced here by renders of example IB/01 at $t = 0.12$ s with VisIt's Subset plot of the levels, which chapter 26 shows how to make.

WHAT ADAPTIVITY SAVES

Example	Levels, ratio	Finest grid if uniform	Cells in the hierarchy
IB/01 rubber band	4, ratio 4 (8 → 512)	$512^2 = 262{,}144$	$16{,}528$ (16 times fewer): 64 + 576 + 2304 + 13,584 on levels 0 to 3
IB/03 tethered leaf	3, ratio 4 (32 → 512)	$512^2 = 262{,}144$	$4{,}608$ (57 times fewer): 1024 + 1024 + 2560
IB/08 glycocalyx (3D)	3, ratio 4	$64 \times 64 \times 512 = 2.1 \times 10^6$	$161{,}280$ (13 times fewer)

The counts come from the VisIt files of the runs described in the READMEs (the `summary.samrai` file of a dump lists every patch).

> VisIt's **Subset** plot of `levels` (chapter 26) shows you your own hierarchy; if the finest level covers most of the box, you are paying for a uniform grid with extra bookkeeping, and fewer levels would be faster.

WHAT ADAPTIVITY COSTS

- The work per step is roughly proportional to the number of cells (plus the cost of the structure and of the solver's iterations), so a hierarchy with a tenth of the cells of the uniform fine grid is about ten times faster.
- The saving grows with the dimension and with how little of the box the structure occupies: that is why 3D is affordable at all. The glycocalyx run (IB/08) would take 13 times longer on a uniform $64 \times 64 \times 512$ grid.
- The price: regridding costs time (every regrid rebuilds the patches, copies the data and re-projects the pressure), and the fine patches must stay ahead of the structure, which is what `regrid_cfl_interval` and `tag_buffer` arrange.
- The far field is computed on coarse cells: a vortex that leaves the refined region is smeared when it crosses into the coarse level. If the wake matters, turn on vorticity tagging, or add a wide `tag_buffer`, and check that the answer does not change (chapter 16).

RUNNING IN PARALLEL

- The patches are distributed over the MPI processes (`mpirun -np 4`); the `LoadBalancer` block decides how (`bin_pack_method = "SPATIAL"`).
- Each process holds the structure points that lie in its patches, plus ghost copies near the patch edges, which is why the force-spreading and velocity-interpolation loops in the update functions walk both local and ghost nodes (IB/04, IB/06, IB/07).

> The adaptive hierarchy and the parallel decomposition are both handled by SAMRAI underneath IBAMR; you choose the levels, the ratio and the patch sizes in the `GriddingAlgorithm` block and the number of processes on the command line, and IBAMR does the rest.

RUNNING IN PARALLEL: OUTPUT, RESTARTS AND HOW MANY CORES

- The output is written per process (`visit_number_procs_per_file`), and a restart needs the same number of processes as the run that wrote the restart files (chapter 3 shows the error you get otherwise).
- Small problems do not scale: the rubber band runs at the same speed on 4 and on 8 cores (10.5 against 10.2 ms per step), because communication and the solver's fixed costs dominate. Chapter 3's "How many cores?" gives the rule of thumb: a few tens of thousands of cells per process.
- The `smallest_patch_size` in the gridding block (32 cells here) sets the smallest piece of work a process can be given; with 16,528 cells in the rubber band's hierarchy and 28 patches on the finest level, 4 processes already leave some idle part of the time.
- 3D runs scale better: IB/08's 161,280 cells on 4 cores took 0.05 s per step, and the flow tank's larger hierarchy would use 8 to 16 cores well. Chapter 12 returns to this with the 3D examples.

THE INPUT KEYS BEHIND THIS CHAPTER: THE INTEGRATORS

```
IBHierarchyIntegrator {
   dt_max               = DT        // the time step
   grow_dt              = GROW_DT   // growth after a reduction
   num_cycles           = 1
   regrid_cfl_interval = 0.5        // regrid after 0.5 cell of motion
   error_on_dt_change   = TRUE      // abort instead of changing DT
   tag_buffer           = 1         // extra cells refined around points
}
IBMethod {
   delta_fcn = "IB_4"               // the kernel
}
INSStaggeredHierarchyIntegrator {
   cfl                            = 0.3
   dt_max                         = DT
   convective_time_stepping_type = "ADAMS_BASHFORTH"
   convective_op_type             = "PPM"
   using_vorticity_tagging        = FALSE
   vorticity_rel_thresh           = 0.25,0.125
}
```

The IB integrator owns the time step and the regridding; the fluid integrator owns the CFL check, the convective scheme and the vorticity tagging. Both get `dt_max`, and they must agree.

THE INPUT KEYS BEHIND THIS CHAPTER: THE GRID AND THE STRUCTURE

```
GriddingAlgorithm {
   max_levels = MAX_LEVELS         // 3 or 4 here
   ratio_to_coarser { level_1 = 4,4  level_2 = 4,4 ... }
   largest_patch_size  { level_0 = NFINEST, NFINEST }
   smallest_patch_size { level_0 = 32, 32 }
   efficiency_tolerance = 0.85e0
   combine_efficiency   = 0.85e0
}
StandardTagAndInitialize {
   tagging_method = "GRADIENT_DETECTOR"
}
IBStandardInitializer {
   max_levels = MAX_LEVELS
   structure_names = "rubber_band_512"
   rubber_band_512 { level_number = MAX_LEVELS - 1 }
}
```

- `efficiency_tolerance`: a patch is accepted when at least 85% of its cells are tagged; `combine_efficiency`: when to split a patch into smaller ones. `smallest_patch_size` and `largest_patch_size` bound the patches.
- Chapter 3 explains the file's structure; chapter 7 the boundary-condition blocks; chapter 6 the structure files.

RULES OF THUMB

Choice	Rule, and where it comes from
Point spacing	$\Delta s \approx \Delta x/2$ on the finest level for IB; elements about Δx across for IBFE (slide 17)
Kernel	`IB_4`; wider only together with coarser point spacing (slide 15)
Thin features and gaps	at least 4 to 8 cells, or they do not exist for the fluid (slide 16)
Time step	start from the example's value; halve until the run is stable, then check that halving again changes nothing you care about (slides 27–28)
Stiffness	no stiffer than the physics needs: every factor of 4 in stiffness costs a factor of 2 in the time step
Grid	refine until the quantity of interest stops changing; report the grid (slides 22, 25, 30)
Levels	3 or 4 levels with ratio 4; the finest level only around the structure and its wake (slides 31–33)
Box	large enough that the walls do not change the answer (chapter 7)
Before a long run	a short run with a few steps, then a look at the output in VisIt (chapter 26)

> Every rule here was a lesson from a run that went wrong. The examples' READMEs record the ones that went wrong on purpose, with IBAMR's exact messages.

WHAT CAN GO WRONG

Symptom	Cause and what to do
`Time step size change encountered` after a few steps	the forcing instability: smaller `DT`, softer springs, slower motion (slides 27–28)
`Internal RefineSchedule<DIM> error` after a few steps (3D)	the same instability, caught by a regrid first (IB/08)
Fluid leaks through the structure	points farther apart than $\Delta x/2$, or elements larger than Δx: regenerate the structure for the grid (slide 17)
Two close structures behave as one	they are within the kernel's width: a wider gap or a finer grid (slide 16)
The answer changes when you refine	not converged: refine again, or accept and state the grid (slides 22, 30)
Nothing moves, or forces look wrong, after changing `MAX_LEVELS`	the structure is assigned to a level that no longer exists or is not the finest: check `level_number` (slide 31)
Very slow steps, many regrids in the log	`regrid_cfl_interval` too small for a fast motion, or a finest level that covers the whole box (slide 33)
Small jitter of a point at rest	the kernel's grid dependence: a wider kernel (`IB_6`) smooths it (slide 15)

Chapter 16 is the full troubleshooting chapter, with the convergence test as its first tool.

SUMMARY

1. An IB simulation solves the Navier–Stokes equations on a fixed grid with a force $\mathbf{f}$ from an elastic structure, and moves the structure with the fluid; the two are coupled by regularized delta functions that spread forces and interpolate velocities with the same kernel.
2. The kernel is a few cells wide, so points should be about $\Delta x/2$ apart and nothing thinner than about 4 cells exists for the fluid.
3. Convergence, order and stability: the two demo scripts show each; the IB method is first order near a thin boundary, so refine until the answer stops changing.
4. The time step is limited by the stiffest element, not by the CFL condition; the limit shows up as “Time step size change encountered”.
5. IBAMR refines the grid only around the structure, keeps the structure on the finest level, and regrids as it moves; the keys are in the integrator, gridding and initializer blocks.

Next, chapter 6: the structure files (`.vertex`, `.spring`, `.beam`, `.target` and the rest), and the generators that write them.

WHAT CHANGED SINCE THE OLDER COURSE NOTES

- The accuracy statement is corrected: first order near a thin interface, second order for smooth problems (Griffith and Peskin 2005), instead of “second order if smooth boundaries”.
- The time-step discussion is rebuilt from measurements on the examples (slide 27) and from IBAMR’s actual logic: `cfl`, `dt_max`, `error_on_dt_change`, and why the CFL message is the symptom of a forcing instability.
- “Two points per grid cell” is kept for IB and stated in element sizes for IBFE, with the numbers of the examples and of IBAMR’s own `ex4`.
- The time stepping is second order with these versions (midpoint rule, Adams–Bashforth), not “first or second depending on the version”.
- The five AMR bullets of the 2014 UNC edition, dropped from the 2020 lecture, are back (slide 32).
- The equation images are typeset; B. Griffith’s 2009 MBI figures (the link is dead) are replaced by renders of example IB/01; the four opening photographs are replaced by a table; IBAMR’s kernel list comes from the 0.19.0 source.
- The demo scripts have Python versions and their bugs are fixed (Appendix A); the “style.visibility” residue and the Sakai download links are gone.

REFERENCES

- C. S. Peskin. The immersed boundary method. *Acta Numerica* 11:479–517, 2002.
- B. E. Griffith and C. S. Peskin. On the order of accuracy of the immersed boundary method: higher order convergence rates for sufficiently smooth problems. *J. Comput. Phys.* 208:75–105, 2005.
- B. E. Griffith, R. D. Hornung, D. M. McQueen and C. S. Peskin. An adaptive, formally second order accurate version of the immersed boundary method. *J. Comput. Phys.* 223:10–49, 2007.
- Y. Bao, J. Kaye and C. S. Peskin. A Gaussian-like immersed-boundary kernel with three continuous derivatives and improved translational invariance. *J. Comput. Phys.* 316:139–144, 2016.
- B. E. Griffith and N. A. Patankar. Immersed methods for fluid–structure interaction. *Annu. Rev. Fluid Mech.* 52:421–448, 2020 (open access: PMC7531444).
- N. A. Battista, W. C. Strickland and L. A. Miller. IB2d: a Python and MATLAB implementation of the immersed boundary method. *Bioinspiration & Biomimetics* 12:036003, 2017.
- R. Courant, K. Friedrichs and H. Lewy. Über die partiellen Differenzengleichungen der mathematischen Physik. *Math. Ann.* 100:32–74, 1928.
- IBAMR: https://ibamr.github.io; SAMRAI: https://computing.llnl.gov/projects/samrai. Kernel names: `ibtk/src/lagrangian/LEInteractor.cpp`; time-step logic: `src/IB/IBHierarchyIntegrator.cpp` and `src/navier_stokes/INSHierarchyIntegrator.cpp`, IBAMR 0.19.0.
- *Fluid Dynamics and IBM Theory*, modules 0-1, 0-2, 1-1, 1-2, 1-3, 1-4: https://github.com/fairyflies9/Fluid-Dynamics-and-IBM-Theory.
- Sources: *0-IBAMR Tutorial: UNC Edition* (May 2014); *Overview of IBM and Numerical Considerations* (MATH 769, January 2018; IBAMR lecture 2, 2020); *Intro to IBM* (2025); the `stiff_ode_example` and `heat_equation` scripts of the courses.